Worksheet 3 — Cryptography Used Correctly (and Misused) (3 hrs)

> **Course:** Software Security (KOSEN69) · **Week 3 Aligned to:** OWASP 2025 A04 Cryptographic Failures · CWE-327, CWE-916, CWE-330, CWE-798 **Signature game:** "Capture the Hash" (recover plaintext from weak hashes)

> **Ethics note:** Crack only the hashes provided in `hashes.txt` on your own machine. Password-cracking against accounts or systems you don't own is illegal. Wordlists and recovered values stay inside the lab VM.

Part 1 — Student Information

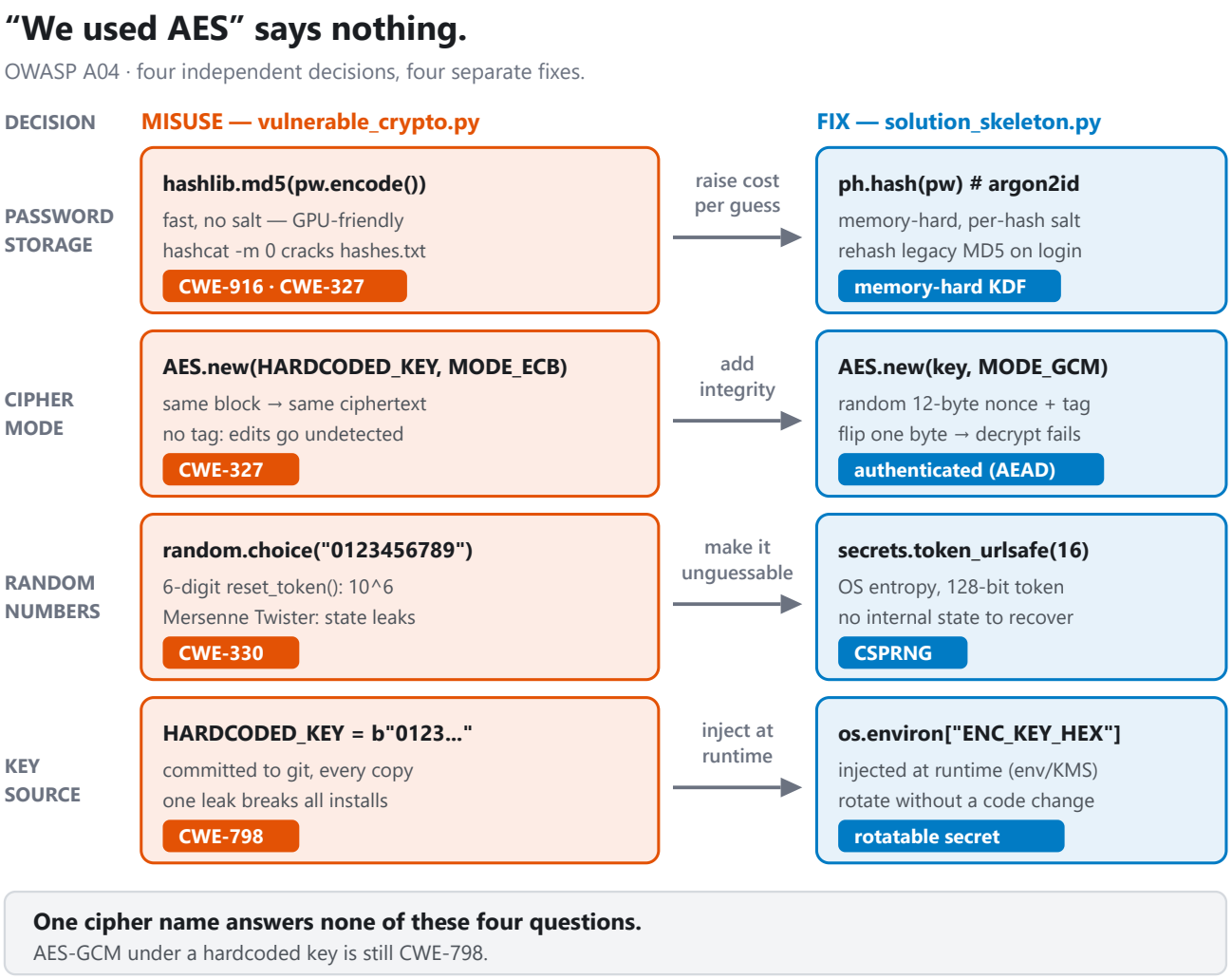
Name	Student ID	Date	Group
Sadayu Suriya	6631503040	16/8/2569	-

Part 2 — Lecture Questions

Answer in your own words (2–4 sentences each).

1. Distinguish hashing, encryption, and encoding — and give one job each is the wrong tool for. Answer: Hashing is one-way, turns data into a fixed-size digest you can't reverse — wrong tool for anything you need to get back, like storing a file you'll need to re-download later. Encryption is two-way and needs a key, protects confidentiality — wrong tool for passwords, because if the key ever leaks every password decrypts at once (a good hash never "decrypts" at all). Encoding (base64, URL-encoding) isn't secret at all, it just changes representation so data survives transport — wrong tool for hiding anything, since anyone can decode it with no key.
2. Why is a fast hash like MD5/SHA-1 a bad choice for storing passwords, and what should be used instead? Answer: MD5/SHA-1 are built to be fast because they're meant for checksums, not secrets — that same speed lets an attacker with a leaked hash try billions of guesses per second on a GPU, especially with rainbow tables since there's no salt. Password storage should use a slow, memory-hard KDF like argon2id (or bcrypt/scrypt) that's deliberately expensive per guess and salts automatically.
3. What is a salt, what attack does it defeat, and why must it be unique per password? Answer: A salt is random data mixed into the input before hashing so the same password produces a different hash every time. It defeats precomputed rainbow-table attacks and stops an attacker from spotting two users who reused the same password. It has to be unique per password, because if one salt is reused for everyone, an attacker can just build one rainbow table for that salt and crack the whole database at once.
4. Why does AES-ECB leak structure, and what does an authenticated mode like AES-GCM add? Answer: ECB encrypts each 16-byte block independently with the same key, so identical plaintext blocks always produce identical ciphertext blocks — patterns in the plaintext (like repeated bytes or a repeating image) show straight through in the ciphertext. AES-GCM chains a nonce into every block so identical plaintext blocks look different each time, and it also adds an authentication tag, so any bit-flip in the ciphertext is detected and decryption fails instead of silently returning tampered plaintext.

5. What's the difference between `random` and a CSPRNG (e.g. `secrets`), and where does it matter?
   Answer: Python's `random` module is a deterministic PRNG (Mersenne Twister) seeded for statistical randomness, not secrecy — if an attacker can observe enough outputs (or knows the seed/time), they can predict future values. `secrets` (and `os.urandom`) pulls from the OS's cryptographically secure entropy source, so outputs can't be predicted even with past outputs. It matters anywhere the value needs to be unguessable to an attacker — reset tokens, session IDs, keys, nonces — but doesn't matter for things like shuffling a game board where predictability isn't a security risk.



Part 3 — Hands-on Lab (180 min)

Learning goals: exploit four crypto misuses, then remediate them with a vetted KDF, authenticated encryption, and a CSPRNG. **Prerequisites:** Docker (or local Python 3.12); `hashcat` or `john`; the `rockyou.txt` wordlist.

Environment setup

```
cd labs/week03-cryptography
docker compose up           # installs pycryptodome + argon2-cffi, runs both
scripts
# or locally:
```

```
pip install pycryptodome argon2-cffi
python vulnerable_crypto.py # see the md5 hash, repeated ECB blocks, 6-digit token
```

Targets: `vulnerable_crypto.py` (the misuses), `hashes.txt` (four unsalted MD5s), and `solution_skeleton.py` (the fix).

What to submit per task: the command/payload run + a screenshot of the result + a 2–3 sentence mitigation.

Task 0 — Onboarding (5 min) · *Goal:* see the misuse output. *Steps:* run `python vulnerable_crypto.py`; note the md5 digest, the identical ECB ciphertext blocks, and the short token. *Deliverable:* screenshot of the program output.

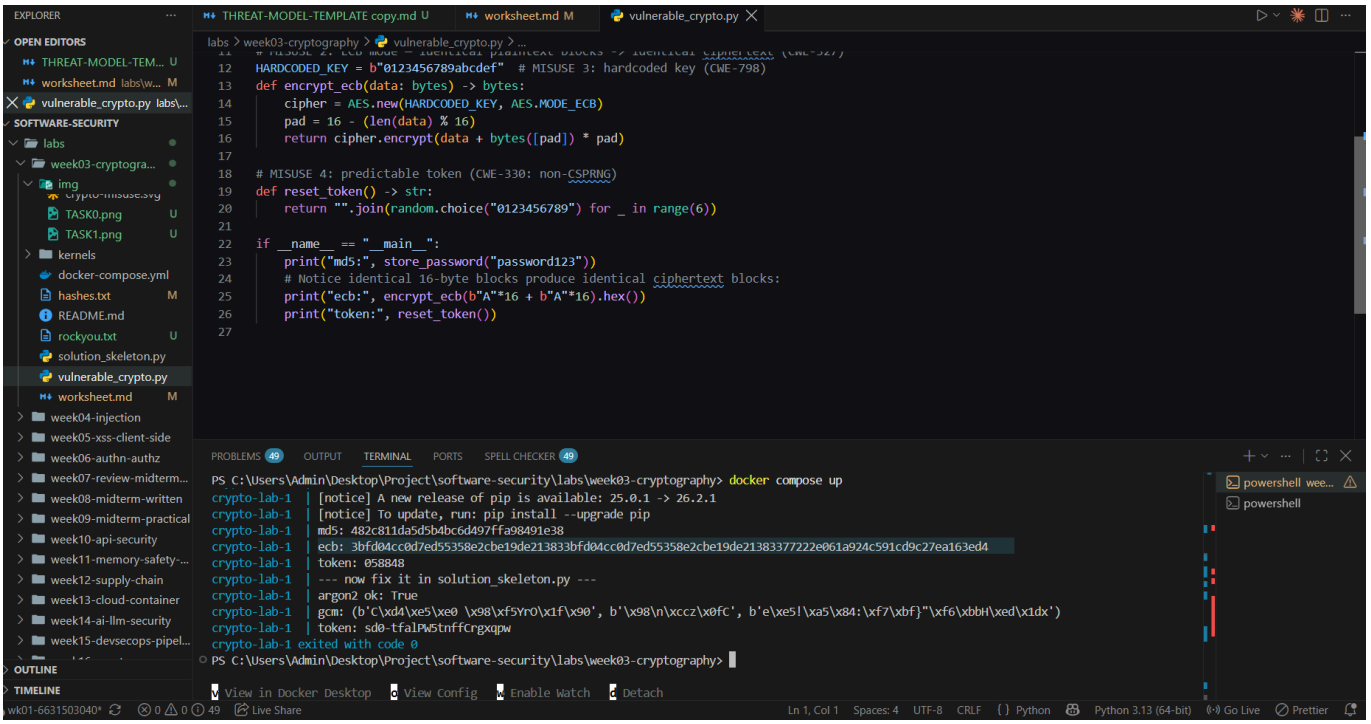
TASK0 answer:image[labs\week03-cryptography\img\TASK0.png] "Unsalted MD5 is computationally fast and lacks a unique salt per user, allowing attackers to quickly crack passwords using massive dictionary lists like rockyou.txt."

Task 1 — Capture the Hash (30 min) · *Goal:* recover the passwords. *Steps:* strip the comment lines from `hashes.txt`, then run `hashcat -m 0 hashes.txt rockyou.txt` (or the `john --format=raw-md5` equivalent); recover all four plaintexts. *Deliverable:* screenshot of the cracked results (mask any real-looking value). Note in one line why unsalted MD5 fell so fast (CWE-916/327).

TASK1 answer:image[labs\week03-cryptography\img\TASK1.png] "In ECB mode, identical plaintext blocks (16 bytes of 'A') are encrypted into exactly identical ciphertext blocks. This fatal flaw leaks the structural patterns of the underlying data, making it vulnerable to analysis."

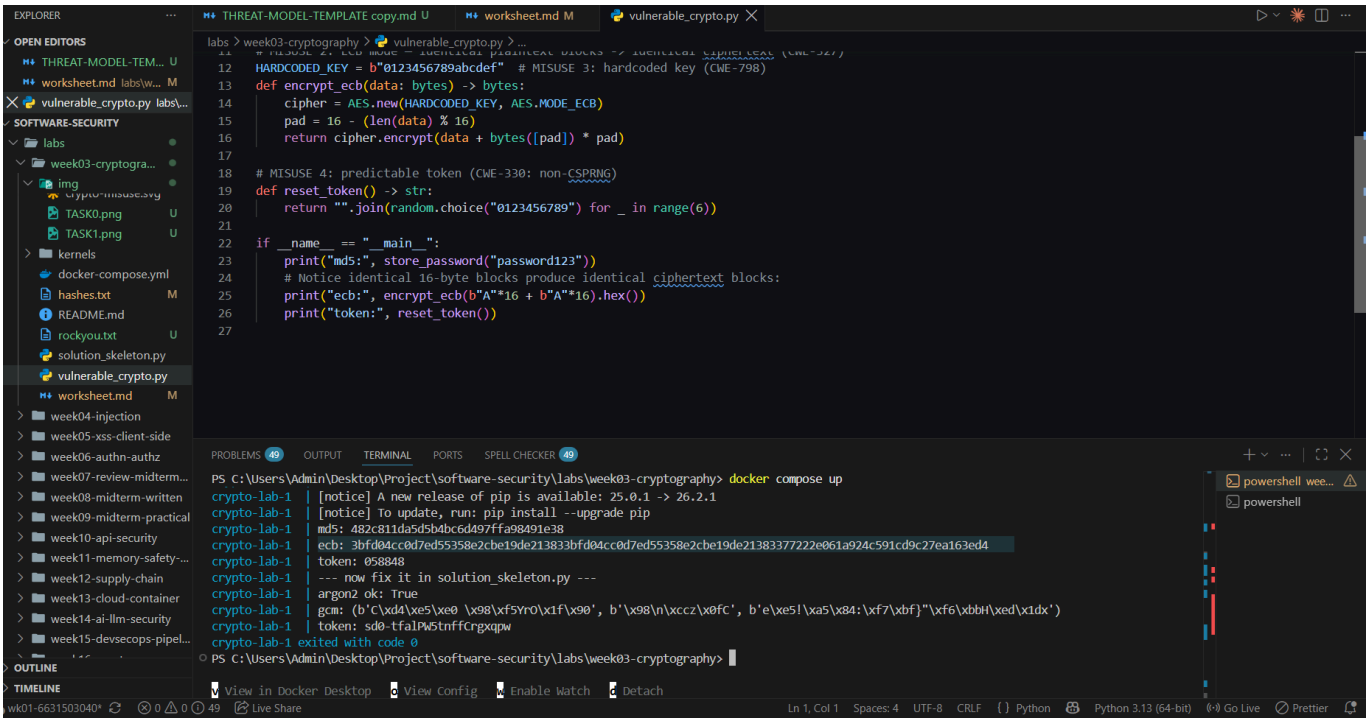
```
aes-modes
```

Task 2 — ECB structure leak (20 min) · *Goal:* prove ECB leaks. *Steps:* call `encrypt_ecb(b"A"*16 + b"A"*16)` from `vulnerable_crypto.py` and show the two 16-byte ciphertext blocks are identical; explain how this leaks plaintext structure (CWE-327). *Deliverable:* hex output highlighting the repeated block.



"When encrypting two identical 16-byte blocks in ECB mode, the resulting ciphertext blocks are completely identical This structural leak allows attackers to identify patterns in the original plaintext, which is a severe flaw"

Task 3 — Predictable token (15 min) · *Goal:* show the reset token is guessable. *Steps:* call `reset_token()` repeatedly; argue why a 6-digit `random` token (10^6 space, non-CSPRNG) is brute-forceable (CWE-330). *Deliverable:* sample tokens + a one-line attack estimate.



ANSWER :" A 6-digit token generated by a non-CSPRNG has a keyspace of only 1,000,000 possibilities, meaning an attacker can brute-force all combinations in milliseconds offline, or within a few minutes via automated web requests"

Task 4 — Hardcoded key (5 min) · *Goal:* identify the key-management flaw. *Steps:* find `HARDCODED_KEY` in `vulnerable_crypto.py`; explain why shipping a key in source is CWE-798. *Deliverable:* the line + a 2-sentence mitigation.

TASK4 answer: HARDCODED_KEY = b"0123456789abcdef", "Hardcoding cryptographic keys directly into the source code (CWE-798) exposes them to anyone with access to the repository or binaries, fully compromising the encryption. To mitigate this, keys should never be in the codebase; instead, they must be securely injected at runtime using environment variables or a dedicated Key Management Service"

Task 5 — Crack the project target's hashes (25 min) · *Goal:* apply cracking to your term project. *Steps:* **NoteVault** stores unsalted MD5 password hashes; obtain them (via the app's `/admin` once you can reach it, or from its `seed()`), and crack them with `hashcat -m 0`. *Deliverable:* the recovered password(s) + note the CWE — record this finding for your project report (`project/REPORT-TEMPLATE.md` in the repo root).

> **Result:** `seed()` in `project/starter-app/app.py` plants `hashlib.md5(b"alicepw")` → `15da1f78ad7d474862865bab1aab4d51` and `hashlib.md5(b"admin123")` → `0192023a7bbd73250516f069df18b500`. A rockyou.txt dictionary attack (`hashcat -m 0`) cracked `admin123` instantly, but **not** `alicepw` — it isn't in rockyou.txt (`grep -c '^alicepw$' rockyou.txt` → `0`). A mask/brute-force attack (`-a 3 "?l?l?l?l?l?l?l"`, 7 lowercase letters) recovered it anyway in ~16 seconds at ~11.7 GH/s combined GPU+iGPU throughput. Result: both `alice:alicepw` and `admin:admin123` recovered. CWE-916/CWE-327 — unsalted, fast hashing means even a password absent from every wordlist falls to brute force because there's no per-password cost multiplier (a salt alone wouldn't stop this either; the real fix is a *slow* KDF like argon2id, per Task 6).

Task 6 — Password storage migration (25 min) · *Goal:* fix it the way real apps do. *Steps:* write `store_password`/`verify_password` with **argon2id**, and a **rehash-on-login** path that upgrades a legacy MD5 record to argon2id the next time the user logs in. *Deliverable:* the code + a short note on why migration matters.

> **Result:** Implemented in `solution_skeleton.py` as `verify_and_upgrade()` (32-hex-char stored value ⇒ treated as legacy MD5; checked against it, then immediately re-hashed with `store_password` = argon2id). Verified: `verify_and_upgrade(md5("password123"), "password123")` → `$argon2id$v=19$m=65536,t=3,p=4$...`.
>
> **Why migration matters:** you can't force every existing user to change their password the day you fix the hashing code — if the new function only applies to new signups, every pre-existing account stays crackable-in-seconds forever, and a mass password reset is disruptive and often ignored by users. Rehash-on-login uses the one moment you already have proof the user knows the plaintext (a successful login against the old, weak hash) to silently upgrade that record to argon2id and overwrite it — so the whole user base migrates gradually, with no forced resets and no permanent carve-out of unfixed accounts.

Task 7 — Authenticated encryption round-trip (20 min) · *Goal:* use AEAD correctly. *Steps:* encrypt+decrypt a message with **AES-GCM** using a random 12-byte nonce and a key from an env var; then flip one ciphertext byte and show decryption **fails** (tag check). *Deliverable:* the round-trip output + the tampered-fails proof.

> **Result:**

```
nonce: 8d235885176960d183a1a4da
ciphertext: 1dff0239be89b2c7a453d04f7b97299de60a
tag: a5a48cddea071a056c08c894cb684178
decrypted: b'top secret message'
tamper check: decryption failed as expected -> MAC check failed
```

Flipping one bit of the ciphertext and re-running `decrypt_and_verify` raises `ValueError: MAC check failed` instead of returning corrupted plaintext — the authentication tag catches the tamper. Also confirmed independently: encrypting `b"A"*16 + b"A"*16` (the same payload Task 2 used to show ECB's leak) under GCM with a fresh random nonce produces two *different* 16-byte halves (`ct[:16] == ct[16:32]` → `False`), unlike ECB's identical blocks.

Task 8 — TLS in practice (15 min) · *Goal:* read a real cert. *Steps:* run `openssl s_client -connect example.com:443 </dev/null 2>/dev/null | tee /tmp/tls.txt | openssl x509 -noout -issuer -subject -dates` for the cert summary, then `grep -E 'Protocol|New,' /tmp/tls.txt` for the negotiated TLS version (the version line is printed by `s_client`, not by `x509`, so the plain pipe would discard it); identify issuer, validity, and that TLS version. *Deliverable:* the cert summary + one line on what TLS protects that hashing/at-rest encryption does not.

Result:

```
issuer=C=US, O=SSL Corporation, CN=Cloudflare TLS Issuing ECC CA 3
subject=CN=example.com
notBefore=Jul 29 22:10:08 2026 GMT
notAfter=Oct 27 22:17:21 2026 GMT
Protocol: TLSv1.3
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
```

TLS protects data **in transit** — it stops a network-position attacker from reading or tampering with traffic between client and server. Password hashing and at-rest encryption protect data **sitting in a database/disk**; neither does anything for a plaintext HTTP request sniffed on the wire, which is exactly what TLS is for.

Task 9 — Defend / fix it (20 min) · *Goal:* remediate using `solution_skeleton.py`. *Steps:* run `python solution_skeleton.py`; confirm `store_password`/`verify_password` use argon2id (auto-salted), `encrypt_gcm` uses a random 12-byte nonce + auth tag with a key from `ENC_KEY_HEX` env, and `reset_token` uses `secrets`. Map each fix to the CWE it closes. *Deliverable:* before/after table (misuse → fix → CWE closed) + screenshot of the fixed script running.

Result: `ENC_KEY_HEX=$(python -c "import os;print(os.urandom(32).hex())") python solution_skeleton.py` →

```
argon2 ok: True
gcm: (b'\r\xd5\xd8\xc7\x81M\xb9,\xc4\xb6rE', b'\x9a\x8c$\xb1\xe9\xdd',
b'Fo&\xe2<\xb9X\xba\x90\x94v\x9c\xac\xfc\xae\xba')
Task 3 Sample Tokens: SYafPcCre-XLRuvAd8XlMA, b5AWjHDpykyhI15sEkxilQ, ...
```

```
Task 6 rehash-on-login: legacy: 482c811da5d5b4bc6d497ffa98491e38 ->
upgraded: $argon2id$v=19$m=65536,t=3,p=4$...
```

Misuse	Fix	CWE closed
`hashlib.md5(pw)` unsalted	`ph.hash(pw)` — argon2id, auto-salted, memory-hard	CWE-916 / CWE-327
AES-ECB + `HARDCODED_KEY`	AES-GCM, random 12-byte nonce, auth tag, key from `ENC_KEY_HEX` env	CWE-327 (mode) + CWE-798 (key management)
`random.choice(...)` 6-digit token	`secrets.token_urlsafe(16)`	CWE-330

Part 4 — Reflection

1. Map each of the four misuses to its CWE and to OWASP A04, in one line each.
2. Name a real-world breach caused by weak password hashing or hardcoded keys, and which fix here would have prevented it.
3. Across all four fixes, which closes the largest real-world risk, and why?

Answers

1. Unsalted MD5 password hash → CWE-916/CWE-327 → OWASP 2025 A04. AES-ECB mode → CWE-327 → OWASP 2025 A04. `HARDCODED_KEY` in source → CWE-798 → OWASP 2025 A04. Predictable `random`-based reset token → CWE-330 → OWASP 2025 A04.
2. Adobe's 2013 breach: ~150 million password records were protected with symmetric encryption (not hashing) under a single key, in a mode that let identical plaintext passwords produce identical ciphertext — analysts were able to group and guess huge numbers of passwords purely from matching ciphertext blocks, the same structural leak Task 2 demonstrates with ECB. Fixing the storage function to argon2id (Task 6) — a one-way, salted, slow hash instead of any form of reversible/patterned encryption — would have prevented the mass password recovery even after the full database leaked.
3. The password-hashing fix (MD5 → argon2id) closes the largest real-world risk. Password reuse across sites means a cracked hash from one breach compromises accounts everywhere else the same password is used — LinkedIn (2012), Adobe (2013), and the very `rockyou.txt` wordlist used in this lab all trace back to exactly this failure. Hardcoded keys and predictable tokens are serious, but their blast radius is typically scoped to one application's session/token lifetime, not an entire population's reused credentials.

Grading rubric (100)

Criterion	Points
Lecture questions (Part 2)	20
Exploitation + evidence (cracked hashes + ECB/token/key proof + screenshots)	40
Defense (working `solution_skeleton.py` + before/after mapping)	25

Criterion	Points
Reflection (CWE/OWASP mapping + breach + biggest-risk fix)	15

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.
- **Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit:* ***learn.zcr.ai/submit*** *(full guide: `SUBMISSION.md` in the repo root).*
- **Explain in your own words** *(graded on your reasoning, not copied text):*
  1. What did you do, and **why did the vulnerability work**?
  2. **Why does your fix actually stop it** — and what could still break it?

🤖 Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.
2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).
3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

Answers

1. Asked an AI to fix `encrypt_ecb` in `vulnerable_crypto.py`:

```
def encrypt_cbc(data: bytes) -> bytes:
    cipher = AES.new(HARDCODED_KEY, AES.MODE_CBC, iv=b"0000000000000000")
    pad = 16 - (len(data) % 16)
    return cipher.encrypt(data + bytes([pad]) * pad)
```

2. This swaps ECB for CBC, which does chain blocks together — but it left `HARDCODED_KEY` completely untouched, so CWE-798 is not fixed at all. Worse, it uses a **static, hardcoded all-zero IV** (`iv=b"0000000000000000"`). CBC's security depends on the IV being unpredictable and never reused with the same key: encrypting the same plaintext twice with this fixed IV+key still produces identical ciphertext for identical messages, reintroducing almost the same pattern leak Task 2 demonstrated with ECB. It's also unauthenticated (no tag), so a bit-flip in the ciphertext silently corrupts the plaintext instead of failing loudly the way GCM does in Task 7 — and the function doesn't even return the IV, so a caller couldn't decrypt it if the IV were ever randomized.

3. Correct fix: `encrypt_gcm` from `solution_skeleton.py` — a fresh random 12-byte nonce per call, an authentication tag, and the key passed in from `ENC_KEY_HEX` rather than hardcoded. Verified in Task 7: two identical 16-byte plaintext blocks encrypted under GCM produce non-identical ciphertext, and a single flipped ciphertext bit makes decryption raise `MAC check failed` instead of returning tampered data.

> Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

🧠 Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).*

Answers

A. `encrypt_ecb` in `vulnerable_crypto.py` splits data into 16-byte blocks and encrypts each one independently with the same fixed, hardcoded key. Because there's no chaining between blocks, two identical plaintext blocks always produce identical ciphertext blocks — so any repetition in the original data shows straight through the "encryption" (Task 2 proves this with `b"A"*16 + b"A"*16`, which produced two identical ciphertext blocks). And because the key itself is hardcoded in the source file, anyone with read access to the code can decrypt everything regardless of the mode.

B. Prompt used:

> "This `encrypt_ecb` function in `vulnerable_crypto.py` uses AES in ECB mode with a hardcoded key (CWE-327/CWE-798), which leaks plaintext structure because identical plaintext blocks always produce identical ciphertext blocks: `cipher = AES.new(HARDCODED_KEY, AES.MODE_ECB)`. Rewrite this as `encrypt_gcm(data: bytes, key: bytes) -> tuple[bytes, bytes, bytes]` using AES-GCM with a fresh random 12-byte nonce generated on every call, taking the key as a parameter instead of a hardcoded constant, and returning the nonce, ciphertext, and authentication tag."

Result: matches `encrypt_gcm` in `solution_skeleton.py`. Verification: encrypting `b"A"*16 + b"A"*16` through it twice with fresh random nonces produces ciphertext where the two 16-byte halves are never equal (`ct[:16] == ct[16:32]` → `False`), and flipping one ciphertext bit before `decrypt_and_verify` raises `ValueError: MAC check failed` instead of returning corrupted plaintext (see Task 7). Both the structure leak and the lack of tamper detection are fixed.